Smart Voice Home Assistant

Raspberry Pi + OpenAI + OLED + GPIO Control

VOICE | OPENAI | OLED | GPIO | PYTHON

Final Project Presentation

By: Trung Nguyen

OVERVIEW

Project Overview

- **Voice-activated home assistant**

  A Raspberry Pi device that waits for a wake command, starts an AI conversation, and gives spoken feedback.

- **Embedded hardware focus**

  The project uses a voice sensor, OLED display, and GPIO-connected outputs.

- **Documentation goal**

  The final GitHub project is organized like a complete engineering report.

OBJECTIVES

Objectives and Goals

- Build a working Raspberry Pi voice assistant prototype.
- Use a dedicated recognition sensor for wake and reset commands.
- Integrate OpenAI transcription, response generation, and TTS.
- Display live status on OLED.
- Control fan, red light, and green light through GPIO.
- Document design process, issues, results, and future improvements.

COMPARISON

Consumer vs. Custom AI Assistant

Typical Consumer Assistant

- Closed ecosystem and limited hardware access.
- Hard to customize GPIO behavior.
- Usually depends on fixed vendor features.
- Little visibility into the internal logic.

This Raspberry Pi Build

- Open Python code and clear documentation.
- Wake command, OLED feedback, and physical GPIO outputs.
- OpenAI runtime plus local AI experiments.
- Expandable for relays, lights, and sensors.

GOAL

Our Goal: Combine Both Worlds

- **Low cost**
  Use Raspberry Pi hardware and common modules.
- **Small footprint**
  Prototype can become a compact desktop assistant.
- **AI conversation**
  Use OpenAI for natural spoken responses.
- **Physical control**
  Connect the assistant to fan and light indicators.
- **Visible feedback**
  OLED shows what the assistant is doing.

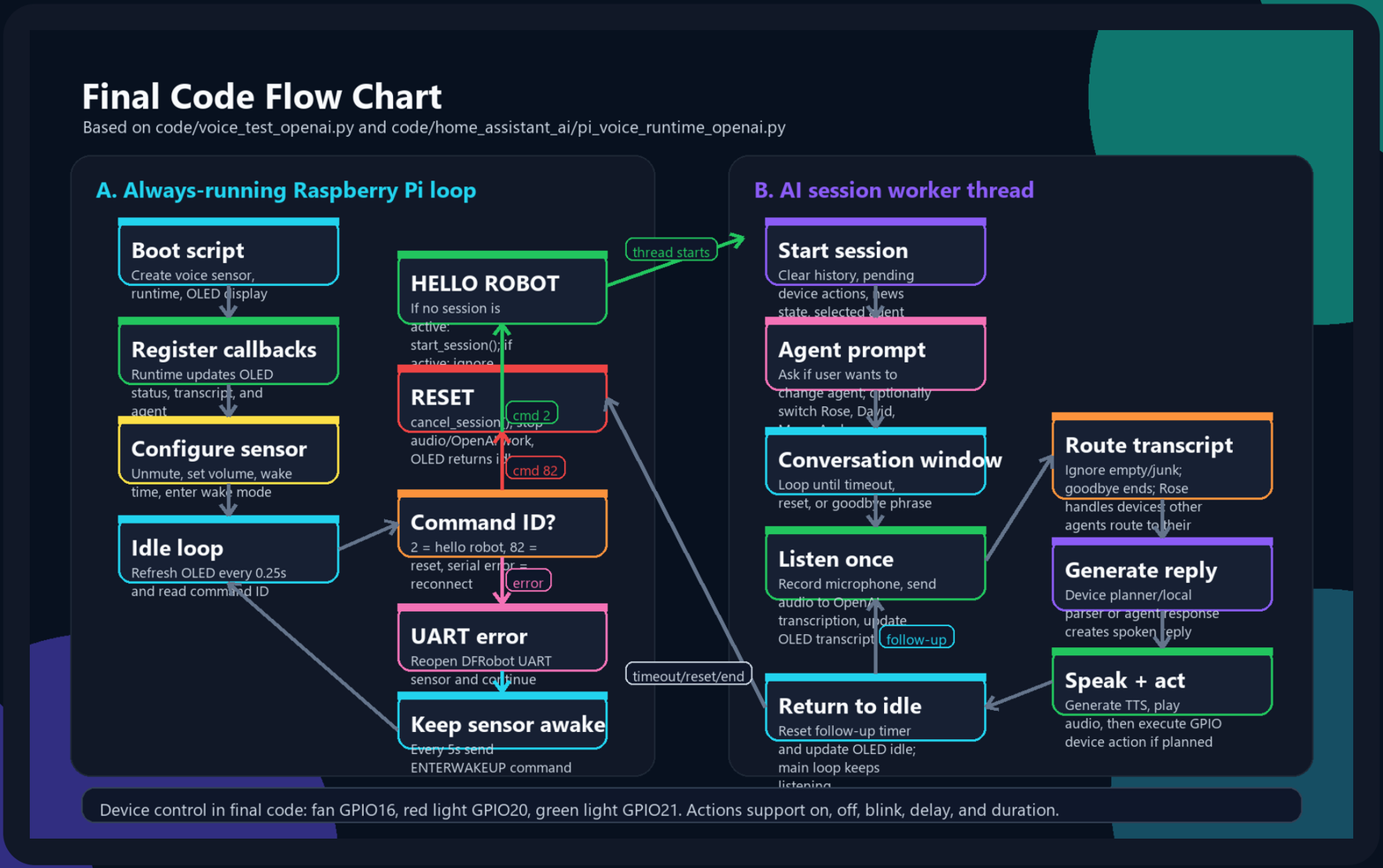
DESIGN

Design and Development

- **Wake layer**
  DFRobot voice recognition sensor detects command IDs.
- **Runtime layer**
  Python starts and cancels threaded assistant sessions.
- **AI layer**
  OpenAI handles transcription, replies, and TTS audio.
- **Output layer**
  OLED, speaker audio, and GPIO provide user feedback.

FLOW CHARTS

Final Code Flow Chart



Corrected from the final code: main loop reads the voice sensor and GPIO actions; the runtime worker handles AI conversation, TTS, OLED updates, and

CODE FLOW

Main Loop Behavior

- **Idle loop**

  OLED refreshes every 0.25 seconds and the DFRobot sensor is read continuously.

- **Hello robot**

  Command ID 2 starts a session only if no session is active.

- **Reset**

  Command ID 82 cancels active audio/OpenAI work and returns OLED to idle.

- **Serial recovery**

  UART read errors reopen the voice sensor and continue.

RUNTIME

AI Session Worker

- **Session start**

  Clears history, pending device actions, agent switch state, and news state.

- **Agent prompt**

  Asks whether to switch agent, then can select Rose, David, Maya, or Andrew.

- **Conversation loop**

  Records microphone, transcribes, routes transcript, replies, speaks, and waits for follow-up.

- **Exit conditions**

  Reset, timeout, goodbye phrase, empty reply, failed TTS, or cancellation.

CODE

Final Main Code Flow

```
voice = DFRobot_DF2301Q_UART()
runtime = PiVoiceRuntimeOpenAI()
oled = OLEDStatusDisplay(...)
cmd_id = voice.get_CMDID()
if cmd_id == HELLO_ROBOT_CMDID:
    runtime.start_session()
elif cmd_id == RESET_CMDID:
    runtime.cancel_session()
```

The main file stays in the sensor loop while the runtime session runs in a worker thread.

GPIO

Device Control Logic

- **Device planner**

  Rose checks state queries, asks AI planner for device commands, then falls back to local parsing.

- **Supported devices**

  Fan on GPIO16, red light on GPIO20, green light on GPIO21.

- **Supported actions**

  On, off, blink, delay, and duration.

- **Execution timing**

  GPIO action runs after the spoken reply finishes.

OLED User Interface

- Agent name
- Agent role
- Network status
- Current action
- Recent user transcript
- Scrolling transcript text for long speech

MECHANICAL

Mechanical Build

- Raspberry Pi is the main control board.
- USB microphone captures user speech during active sessions.
- OLED is placed where the user can see system state.
- Voice recognition sensor is positioned for wake/reset commands.
- Final enclosure CAD is planned after the prototype layout is locked.

ELECTRICAL

Electrical Design

- **UART voice sensor**

  Pin 8 / GPIO14 transmits commands. Pin 10 / GPIO15 receives sensor data.

- **OLED display**

  I2C display at address 0x3C using luma.oled.

- **GPIO outputs**

  Fan GPIO16, red indicator GPIO20, green indicator GPIO21.

- **Safety**

  GPIO should control driver or relay circuits for larger loads.

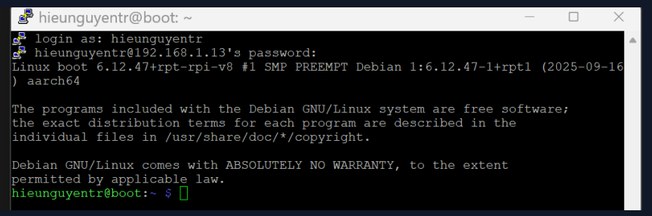
EXPERIMENTS

Local AI Experiments

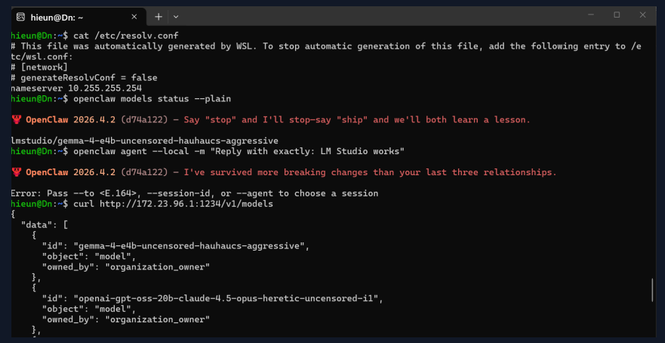
- **Vosk**
  Installed and tested for offline speech recognition.
- **Whisper.cpp**
  Explored for local speech-to-text on Raspberry Pi.
- **Piper**
  Explored for text-to-speech voices.
- **Ollama / Qwen / OpenClaw**
  Tested as local or action-agent AI features.
- **Low-RAM lesson**
  On Raspberry Pi 3, heavy AI steps must run carefully and not all at once.

PROGRESS

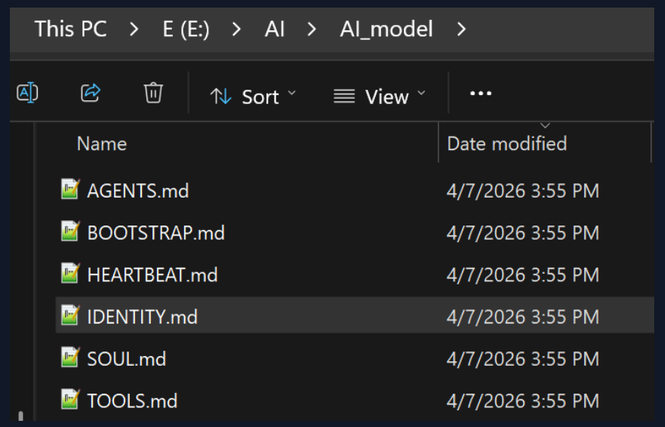
Development Evidence



Raspberry Pi remote access



OpenClaw setup



File automation workflow

TESTING

Results

Works
Wake command starts session

Works
OLED status callback updates

Works
OpenAI transcription and response

Works
Reset cancels active session

Tested
Vosk and microphone debugging

Tested
Low-RAM local AI experiments

LESSONS

Issues and Lessons Learned

- **Microphone setup required debugging**
  USB audio detection worked, but input device selection needed careful testing.
- **Vosk was sensitive to noise**
  Offline recognition was fast but picked up background sound.
- **Low RAM changed the design**
  After moving to Raspberry Pi 3, AI tools had to run more carefully and sequentially.
- **Test hardware one part at a time**
  Sensor, OLED, GPIO, microphone, and AI runtime each needed isolated tests.

BOM

Bill of Materials

Component	Purpose
Raspberry Pi 3 / 4	Main embedded computer
DFRobot DF2301Q	Wake and reset command sensor
USB lavalier microphone	Speech input
OLED display	Status and transcript feedback
Fan / LEDs / relay wiring	Physical device control
microSD + HDMI + VNC/SSH	Setup and remote development

DOCUMENTATION

GitHub Project Report

- Home page with project overview and architecture.
- Flow charts, BOM, mechanical build, electrical, results, code, progress reports, and CAD sections.
- Clean public code folder with requirements and service template.
- Private OpenAI API keys and Wi-Fi credentials are excluded.

FUTURE WORK

Future Improvements

- Record and publish final demo video.
- Finalize enclosure CAD and physical mounting.
- Add exact BOM prices and purchase links.
- Improve microphone noise handling.
- Add more device-control commands and safety checks.
- Package the project for easier installation on a new Raspberry Pi.

FINAL

Conclusion

- The project successfully demonstrates a Raspberry Pi voice assistant prototype.
- The assistant connects wake detection, OpenAI responses, OLED feedback, and GPIO device control.
- The corrected flow chart now matches the final code.
- Next steps are demo video, exact BOM, enclosure CAD, and install polish.